

Lab 9 – Designing a Simple Egg Timer Using Vivado 2024.1 and NEXYS A7

Cole Foster (101230199)

Board Model: XC7A100TCSG324-1
Programming Language: Verilog 2016

ELEC3500 2025-04-05

Contents

Introduction and Design Goals	2
Design Requirements	2
Design Solutions and Implementation	4
.....	4
Clocking.....	4
Debounce	5
Timer On.....	5
Count Time	6
7-Segment Decoder/MUX	7
7-Segment Display Driver	9
Master Controller	10
Innovative and Alternative Solutions.....	12
Observations	15
Conclusion	15
Acronyms and Terms	16
Appendix 1: No Innovation Egg Timer Full Verilog Code Files.....	17
.....	17
Appendix 2:Innovation Egg Timer Full Verilog Code Files	20

Introduction and Design Goals

In this lab, concepts from previously completed lab work are compiled to design a simple *egg timer* using procedural and structural Verilog code in Vivado 2024.1. Fundamental principles such as behavioral design, structural design, and synchronous circuit design were learned in the ELEC3500 Verilog labs previously completed. These principles will be applied and built upon to design and innovate the FPGA egg timer. The design and implementation section of this report includes detailed systems diagrams of all components. This section includes a discussion on the Verilog code for each component as well as alternative designs. The innovation section of the report includes details on components added to the design for additional functionality. These details include a new system diagram and additional Verilog module discussion. The Observations sections of the report discussed features and the functionality of both the non-innovative and the innovative designs. Improvements and planned patches are discussed in detail in the observations as well. The conclusion briefly summarizes the results and future ambitions of the lab. Please refer to the acronyms section, for the definitions of italicized key concepts discussed in the lab. Finally, all the code written for this lab has been attached to the first and second appendices of this report.

Design Requirements

The egg timer in this design will be implemented on a NEXYS A7 FPGA board (Exact model: XC7A100TCSG324-1) using Verilog 2016. The figure below shows the high-level system diagram for the egg timer and outlines the basic feature requirements to be met in this design using this specific board.

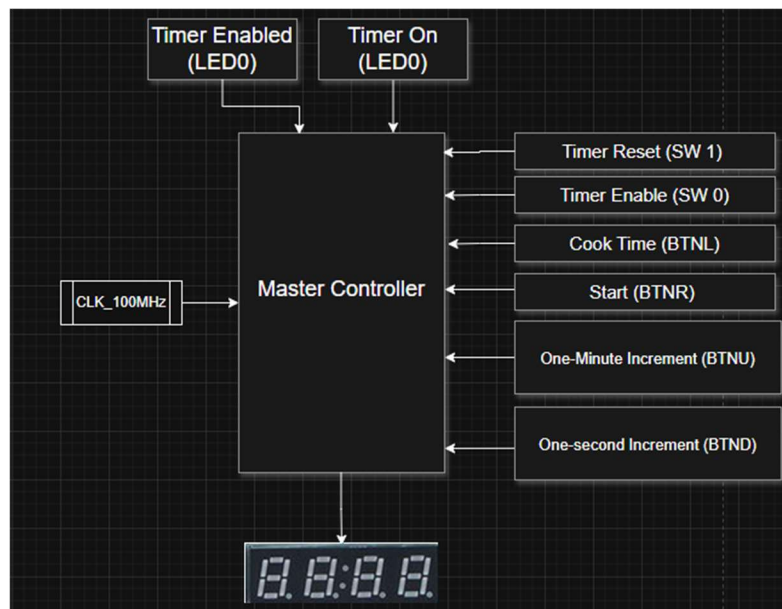


Figure 1. High-level system diagram outlining basic components and requirements of the non-innovation egg timer design.

The standard egg timer features a four-digit countdown timer on four seven-segment displays as an output and the following inputs: Cook Time (BTNL): This button Enables control of the countdown timer while *HI*. The signal from the button must be debounced. Start Timer (BTNR): Toggles the countdown on a timer between running and stopping. The signal is initially LO upon programming the board or upon the clock expiry of the board. Cook time must be HI for the countdown clock to pause when Pause is HI. Cook Time can be LO to start the countdown. Minute Increment (BTNU): This button increments the minutes of the countdown by one minute.

The countdown must be paused or done, and the Cook Time signal must be HI. The maximum minute count must be 59min. The signal from the button must be debounced. Second Increment (BTND): This button increments the seconds of the countdown by one second. The countdown must be paused or done, and the Cook Time signal must be HI. The maximum second count must be 59sec. The signal from the button must be debounced. Reset (SW1): This button resets the countdown to 00:00 (MM:SS) when HI. Timer Enable (SW0): Enables driving clock signal. When Timer Enable LO, the timer cannot be set or altered.



Figure 2. Four-digit seven-segment LED display for timer. The desired output format for the counter is MM:SS. [0]

Two additional outputs have been added to control and observe the clock; Timer Enabled and Timer On respectively. When Timer Enable is HI, Timer Enabled output will be HI. Timer On strobes, when the countdown is running to signify the clock has been started or has expired. As previously stated, a debounce module will also be necessary to filter out false inputs and *signal bounce* from the buttons associated with setting the timer. The clock speed associated with the seven-segment display should have minimal delay to handle *button spamming* appropriately. The switch signals do not need to be debounced as the signal bounce must stabilize after a short time of being either LO or HI state. The clock speed provided by the board is 100MHz which is much too fast for the clock timer functionality. Therefore, the circuit must be driven by the 100MHz clock provided by the ARTYX7 and stepped down to the appropriate clock speed.

Design Solutions and Implementation

This section discussed the components found on the system diagram below in Fig. 1. The first appendix contains all the code for every file used in the non-innovative egg-timer. Below is a detailed systems diagram mapping all the modules to their respective functionalities in the egg timer design.

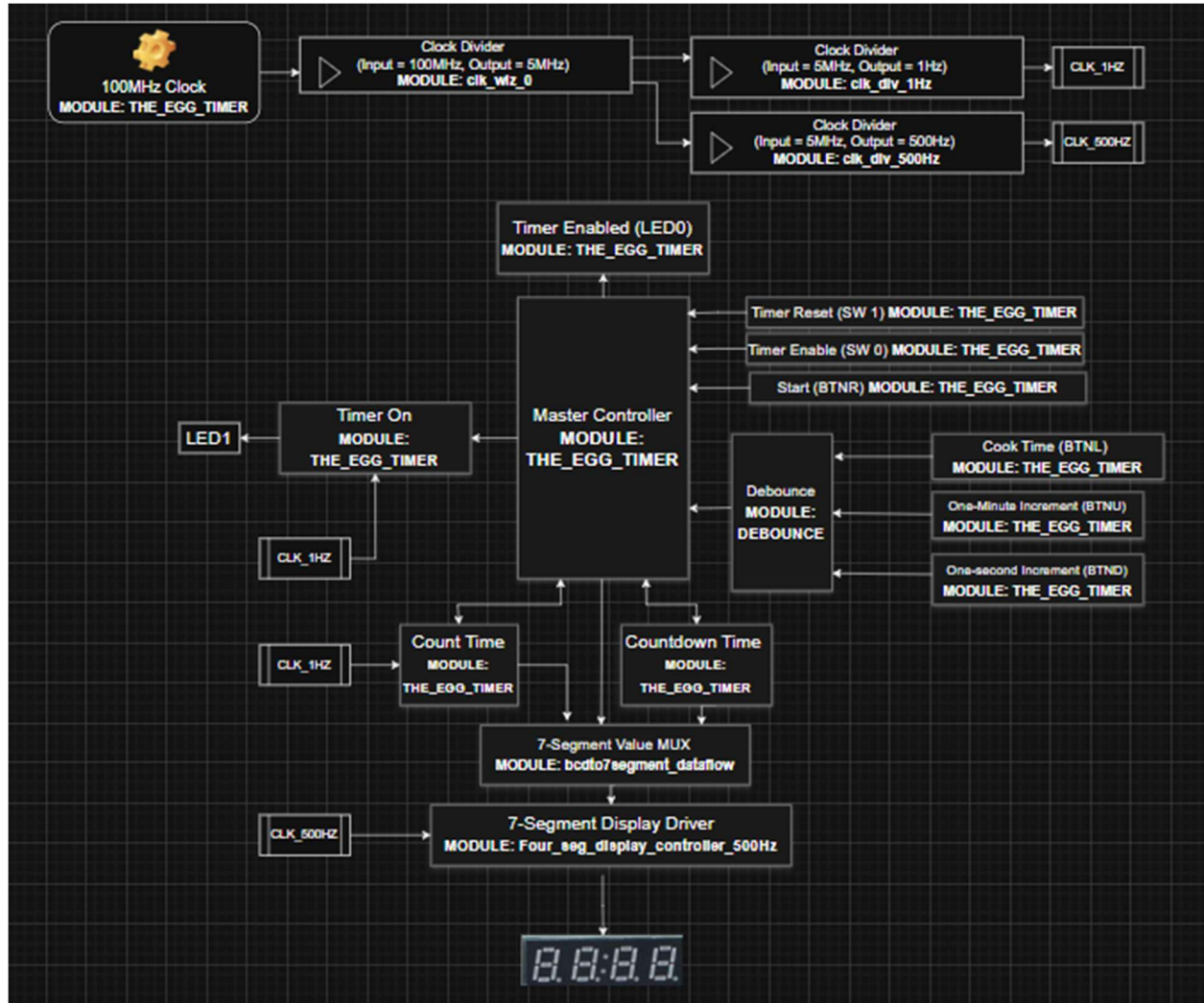


Figure 3. Egg timer system diagram with no innovations made

There are seven key features to this design: system clocking, the master controller, input debouncing, Timer running output, the 7-segment mux to pass seven-bit data through *BCD* encoders to the final portion, the display driver. Additional features include a 6-bit input alpha-numeric decoder as well as a toggle switch for the Seconds Increments button to switch between one and ten-second increments. The system diagram designed with innovative features is shown below in Fig. 20. All these components and their code are discussed in the following sub-sections of this report. The complete Verilog modules can be found in the first appendix of this report.

Clocking

Four clocks are used in this design: a 100MHz clock, a 5MHz clock, a 500Hz clock, and a 1Hz clock. Each clock has a unique function and Verilog module in the mode. 100MHz clock is an input signal provided by the XC7A100TCSG324-1 board and is too high a frequency for this timer. Therefore, it must be broken down to the appropriate frequencies. The 100MHz is first stepped down to 5MHz on line 45 of Fig. 3 below using the Vivado 2024.1 clock wizard. This step down lessens the resources used in the following behavioral dividers and avoids the timing constraints associated with high frequencies in slow-

running logic circuits. The 5MHz signal is then stepped down to 500Hz and 1Hz to drive the display and the clock on the display respectively

```

44 : // Convert 100MHz to 5MHz
45 : clk_wiz_0 fiveMHz(.clk_out1(clock5MHz),.reset(pauseTimer), .clk_in1(clock100MHz));
46 :
47 : // Convert 5MHz to 1Hz
48 : clk_div_1Hz oneHz(.clk_5MHz(clock5MHz),.reset(pauseTimer), .clk_out_1Hz(clk1Hz));
49 :
50 : //Convert 5MHz to 500Hz
51 : clk_div_500Hz Clk500Hz(.clk_5MHz(clock5MHz), .clk_out_500Hz(clk_500Hz));

```

Figure 4. Code snippet of Verilog File Egg_Timer.v initializing the “THE_EGG_TIMER” module and its inputs, outputs, connecting wires, and registers for the clocks and countdown.

Debounce

The debounce module eliminates potential false input signals by counting the amount of time starting a new value is input and allowing said new signal through to the output if it is stable for a minimum amount of time. This debounce ensures the input signals are stable for a minimum of 10ms by counting 50000 5MHz clock cycles. This is a much faster frequency than people can generate input signals by button spamming, so there is no concern about filtering out input signals by accident.

$$\text{Number of cycles} = \text{Time} \times \text{Clock Frequency}$$

$$0.01s \times 5,000,000\text{Hz} = 50,000 \text{ cycles}$$

```

23 : module Debounce(
24 :     input clock5MHz,
25 :     input btn_pressed,
26 :     output reg btn_debounce
27 : );
28 :     reg store_value;
29 :     reg [22:0] high_count, low_count;
30 :
31 :     always@(posedge clock5MHz) begin
32 :         store_value <= btn_pressed;
33 :         if(high_count == 50000 || low_count == 50000) begin
34 :             btn_debounce <= store_value;
35 :             high_count <= 0;
36 :             low_count <= 0;
37 :         end else if(btn_pressed)
38 :             high_count <= high_count + 1;
39 :         else
40 :             low_count <= low_count + 1;
41 :     end
42 : endmodule
43 :

```

Figure 5. Code snippet of Verilog File Debounce.v initializing the “Debounce” module and its inputs, outputs, connecting wires, and registers.

The code snippet below in Fig. 6 shows the Debounce module being called and mapped to the Minutes, Seconds, and Cook Time buttons. The Start Clock/Pause button does not need a debounce signal as it is a constant state clock output (i.e. either counting down or pausing) and to pause the clock, the Cook time button needs to be HI as well as the Start Clock button. This reduces the chance of false inputs because the output signal must be stabilized by the debounced Cook Time signal and the fact that multiple buttons need to be pressed at the same time when the clock is running.

```

76 : //Debounce the minutes, seconds, and cookTime buttons
77 : Debounce mins(.clock5MHz(clock5MHz), .btn_pressed(btnMinutes), .btn_debounce(minutesEN));
78 : Debounce secs(.clock5MHz(clock5MHz), .btn_pressed(btnSeconds), .btn_debounce(secondsEN));
79 : Debounce cookTime(.clock5MHz(clock5MHz), .btn_pressed(btnCookTime), .btn_debounce(cookTimeEN));

```

Figure 6. Code snippet of Verilog Module “Debounce” being used to debounce the minutes and second increments, as well as the cook time buttons.

Timer On

This section of code is responsible for generating an output signal that oscillates between HI and LO at exactly 1Hz frequency if and only if the countdown on the clock has been started. If the clock has expired or is not set, this signal is to remain LO. This is supposed to inform the user if the clock is running without having to check the board. The output signal assigned to the Timer On output is mapped to the

LED1 light on the board. This portion of code was written structurally over behaviourally due to its simplicity.

```
187 | // Output wires to display on LEDS
188 | assign TimerEnable = enable; // LED0
189 | assign TimerOn = (start && enable) ? clk1Hz : 1'b0; // LED1
```

Figure 7. Code snippet of Verilog Module "THE_EGG_TIMER" used to generate the Timer On output signal for LED1

Count Time

This section of code is responsible for two separate tasks: Incrementing the counter to be set and counting down the timer at a 1Hz frequency. The Verilog Code snippet below in Fig. 8 shows the code responsible for incrementing the clock time by one minute when the Minute Increment button is pressed. This code was written behaviourally due to the potential complexity of the alternative assign statements. The output of this snippet is the countm1 and countm2 registers that increment their stored 4-bit values every time the minuteEN signal from the Minute Increment debounce goes to HI.

```
81 | //Increment digits when setting the time. Format = M2:M1:S2:S1
82 | //Incrementing the minutes count
83 | always@(posedge minutesEN or posedge reset) begin
84 |     if(reset)begin
85 |         countm2 <= 0;
86 |         countm1 <= 0;
87 |     end else if(btnCookTime && enable) begin
88 |         // Minute 2 increment
89 |         if(minutesEN && countm2 != 5 && countm1 == 9)
90 |             countm2 <= countm2 + 1;
91 |         else if(minutesEN && countm2 == 5 && countm1 == 9)
92 |             countm2 <= 0;
93 |         //Minute 1 increment
94 |         if(minutesEN && countm1 != 9)
95 |             countm1 <= countm1 + 1;
96 |         else if(minutesEN && countm1 == 9)
97 |             countm1 <= 0;
98 |     end
99 | end
```

Figure 8. Code snippet of Verilog Module "THE_EGG_TIMER" used to increment the minutes count on the egg timer.

The code in Fig. 8 should have taken on the added innovative characteristic of the Seconds Increment button as seen in Fig. 9. The Seconds Increment button has an added switch input to toggle the increment between one second added to the clock time and ten seconds. The simple addition of lines 110-119 discourages the user from spamming the button inputs when setting the time and is a good quality of life change. Due to time restrictions, our team was not able to successfully implement the additional feature to the minute increment code snippet.

```
100 | // Incrementing the seconds count
101 | // Timer format is M2:M1:S2:S1
102 | always@(posedge secondsEN or posedge reset) begin
103 |     if(reset)begin
104 |         counts2 <= 0;
105 |         counts1 <= 0;
106 |     end else if(btnCookTime && enable) begin
107 |         //Second 2 increment
108 |         if(secondsEN && counts2 != 5 && counts1 == 9)
109 |             counts2 <= counts2 + 1;
110 |         else if (secondsEN && counts2 != 5 && countTens)
111 |             counts2 <= counts2 + 1;
112 |         else if (secondsEN && counts2 == 5 && countTens)
113 |             counts2 <= 0;
114 |         else if(secondsEN && counts2 == 5 && counts1 == 9)
115 |             counts2 <= 0;
116 |         // Second 1 increment
117 |         if(secondsEN && counts1 != 9 && !countTens)
118 |             counts1 <= counts1 + 1;
119 |         else if(secondsEN && counts1 == 9 && !countTens)
120 |             counts1 <= 0;
121 |     end
122 | end
```

Figure 9. Code snippet of Verilog Module "THE_EGG_TIMER" used to increment the seconds count on the egg timer. The lines that were changed or added in our group's innovative design have been highlighted for convenience

The following code snippets shown in Fig. 10 and Fig. 11, are responsible for counting down the seconds and minutes on the clock respectively. This logic counts the down the clock with the following format:

M2:M1:S2:S1. The S1 and M1 digits count from 0-9 and the S2 and M2 digits count from 0-5. Therefore the maximum time possibly programmed to count down from is 59 minutes, 59 seconds.

```

124 ⊕ //De-increment when timer started
125 ⊕ //Seconds 1 De-increment
126 ⊕ always @(posedge clk1Hz) begin
127 ⊕     if(activeRST)
128 ⊕         countdowns1 <= 4'b0000;
129 ⊕     else begin
130 ⊕         if(!start && !pause)
131 ⊕             countdowns1 <= counts1;
132 ⊕         else if(pause)
133 ⊕             countdowns1 <= countdowns1;
134 ⊕         if(start && !countdownDone) begin
135 ⊕             if(countdowns1 == 4'b0000)
136 ⊕                 countdowns1 <= 4'b1001;
137 ⊕             else
138 ⊕                 countdowns1 <= countdowns1 - 1;
139 ⊕         end
140 ⊕     end
141 ⊕ end
142 ⊕
143 ⊕ //Seconds 2 De-increment
144 ⊕ always @(posedge clk1Hz) begin
145 ⊕     if(activeRST)
146 ⊕         countdowns2 <= 4'b0000;
147 ⊕     else begin
148 ⊕         if (!start && !pause)
149 ⊕             countdowns2 <= counts2;
150 ⊕         else if(pause)
151 ⊕             countdowns2 <= countdowns2;
152 ⊕         if(start && !countdownDone) begin
153 ⊕             if(countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
154 ⊕                 countdowns2 <= 4'b0101;
155 ⊕             else if(countdowns1 == 4'b0000)
156 ⊕                 countdowns2 <= countdowns2 - 1;
157 ⊕         end
158 ⊕     end
159 ⊕ end
160 ⊕

```

Figure 10. Code snippet of Verilog Module "THE_EGG_TIMER" used to de-increment the seconds count on the egg timer.

```

161 ⊕ //Minutes 1 De-increment
162 ⊕ always @(posedge clk1Hz) begin
163 ⊕     if(activeRST)
164 ⊕         countdownm1 <= 4'b0000;
165 ⊕     else begin
166 ⊕         if(!start && !pause)
167 ⊕             countdownm1 = countm1;
168 ⊕         else if(pause)
169 ⊕             countdownm1 <= countdownm1;
170 ⊕         if(start && !countdownDone) begin
171 ⊕             if(countdownm1 == 4'b0000 && countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
172 ⊕                 countdownm1 <= 4'b1001;
173 ⊕             else if(countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
174 ⊕                 countdownm1 <= countdownm1 - 1;
175 ⊕         end
176 ⊕     end
177 ⊕ end
178 ⊕
179 ⊕ //Minutes 2 De-increment
180 ⊕ always @(posedge clk1Hz) begin
181 ⊕     if(activeRST)
182 ⊕         countdownm2 <= 4'b0000;
183 ⊕     else begin
184 ⊕         if(!start && !pause)
185 ⊕             countdownm2 = countm2;
186 ⊕         else if(pause)
187 ⊕             countdownm2 <= countdownm2;
188 ⊕         if(start && !countdownDone) begin
189 ⊕             if(countdownm1 == 4'b0000 && countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
190 ⊕                 countdownm2 <= countdownm2 - 1;
191 ⊕         end
192 ⊕     end
193 ⊕ end

```

Figure 11. Code snippet of Verilog Module "THE_EGG_TIMER" used to de-increment the minutes count on the egg timer.

7-Segment Decoder/MUX

The following Verilog code snippet is shown in Fig. 12 shows the logic for a BCD to 7-segment decoder. It takes a 4-bit binary value 'x' as its input and outputs the 7-segment code to display the on the display. The output of this circuit must still be directed to the correct 7-segment display and therefore requires a

display controller. This is covered in the next section of this report. The process and logic were learned in ELEC3500, Verilog Lab 6, and are designed using behavioral logic to take advantage of the case statement behavior when defining all the decode combinations.

```

22 |
23 | module bcdto7segment_dataflow(
24 |     input wire [3:0] x,
25 |     output reg [6:0] seg
26 | );
27 |
28 | always @ (*) begin
29 |     case (x)
30 |         4'b0000: seg <= 7'b1000000;
31 |         4'b0001: seg <= 7'b1111001;
32 |         4'b0010: seg <= 7'b0100100;
33 |         4'b0011: seg <= 7'b0110000;
34 |         4'b0100: seg <= 7'b0011001;
35 |         4'b0101: seg <= 7'b0010010;
36 |         4'b0110: seg <= 7'b0000010;
37 |         4'b0111: seg <= 7'b1111000;
38 |         4'b1000: seg <= 7'b0000000;
39 |         4'b1001: seg <= 7'b0010000;
40 |         default: seg <= 7'b1111111;
41 |     endcase
42 | end
43 | endmodule

```

Figure 12. Code snippet of Verilog file "sevenSegBCDDisplay" used to decode 4-bit numbers into 7-segment display code combinations.

The Verilog code segment below (Fig. 13) is a portion of the THE_EGG_TIMER module in which the decoders for each digit on the display board are defined.

```

194 | // Binary to 7 segment conversion for the counts module calling
195 | bcdto7segment_dataflow S1(.x(countdowns1), .seg(seg1));
196 | bcdto7segment_dataflow S2(.x(countdowns2), .seg(seg2));
197 | bcdto7segment_dataflow M1(.x(countdownm1), .seg(seg3));
198 | bcdto7segment_dataflow M2(.x(countdownm2), .seg(seg4));

```

Figure 13. Code snippet of Verilog Module "bcdto7segment_dataflow" being initiated inside "THE_EGG_TIMER" four times. Therefore each digit has a decoder.

The following Verilog code snippets, Fig. 14 and Fig. 15 were not fully integrated into the egg-timer due to time constraints, however, the design improves upon Fig. 11 and Fig. 12. In the design below, the decoder has been expanded to output character codes for both number and letters, and the display has been expanded to use all 8 digits. This design also requires a more comprehensive display controller which will be included in the next section.

```

114 // Decoder: 5-bit letter code (A-Z, space) to 7-segment pattern
115 module AlphaTo7Segment (
116     input [6:0] x, // Letter code: 0=A,1=B,...25=Z, 26=blank
117     output reg [6:0] seg // 7-segment output (active-low)
118 );
119     always @(*) begin
120         case (x)
121             // A-Z letters:
122             6'd0: seg <= 7'b1000000;
123             6'd1: seg <= 7'b1111001;
124             6'd2: seg <= 7'b0100100;
125             6'd3: seg <= 7'b0110000;
126             6'd4: seg <= 7'b0011001;
127             6'd5: seg <= 7'b0010010;
128             6'd6: seg <= 7'b0000010;
129             6'd7: seg <= 7'b1111000;
130             6'd8: seg <= 7'b0000000;
131             6'd9: seg <= 7'b0010000;
132
133             //now define teh characters
134             6'd10: seg = 7'b0001000; // A -> segments a,b,c,e,f,g on (shape: A with middle, no bottom)
135             6'd11: seg = 7'b0011111; // B -> segments b,c,d,e,f,g on (approximate 'B', looks like 8)
136             6'd12: seg = 7'b0110001; // C -> segments a,d,e,f on (looks like C)
137             6'd13: seg = 7'b0011011; // D -> segments b,c,d,e,g on (approximate 'D', looks like 0)
138             6'd14: seg = 7'b0110011; // E -> segments a,d,e,f,g on (E)
139             6'd15: seg = 7'b0111000; // F -> segments a,e,f,g on (F)
140             6'd16: seg = 7'b0101111; // G -> segments a,c,d,e,f,g on (G)
141             6'd17: seg = 7'b0011100; // H -> segments b,c,e,f,g on (H)
142             6'd18: seg = 7'b0000110; // I -> segments b,c on (I, shown as l)
143             6'd19: seg = 7'b0011110; // J -> segments b,c,d,e on (J)
144             6'd20: seg = 7'b0010101; // K -> segments b,c,e,f,g on (approximate K as H, 7-seg can't display K distinctly)
145             6'd21: seg = 7'b0110000; // L -> segments d,e,f on (L)
146             6'd22: seg = 7'b1000111; // M -> segments b,c,e,f on (approximate M as two vertical columns)
147             6'd23: seg = 7'b0101011; // N -> segments c,e,g on lowercase (N)
148             6'd24: seg = 7'b0111111; // O -> segments a,b,c,d,e,f on (O, same as 0)
149             6'd25: seg = 7'b0011000; // P -> segments a,b,e,f,g on (P)
150             6'd26: seg = 7'b0011010; // Q -> segments a,b,c,d,f,g on (Q, looks like 9)
151             6'd27: seg = 7'b0011001; // R -> segments a,b,c,e,f,g on (R, appears similar to A on 7-seg)
152             6'd28: seg = 7'b0100101; // S -> segments a,c,d,f,g on (S, like digit 5)
153             6'd29: seg = 7'b0001111; // T -> segments a,b,c on (T, shown as 7)
154             6'd30: seg = 7'b0011110; // U -> segments b,c,d,e,f on (U)
155             6'd31: seg = 7'b0011101; // V -> segments c,e on (V, appears as two bottom corners lit)
156             6'd32: seg = 7'b0011110; // W -> segments b,c,d,e,f on (W, shown as U)
157             6'd33: seg = 7'b0001001; // X -> segments b,c,e,f on (X, shown as two vertical columns like M)
158             6'd34: seg = 7'b0011010; // Y -> segments b,c,d,f,g on (Y)
159             6'd35: seg = 7'b0110010; // Z -> segments a,b,d,e,g on (Z, looks like digit 2)
160
161             // Blank (space):
162             6'd36: seg = 7'b1111111; // all segments off (blank)
163             6'd37: seg = 7'b1101010; // front half of two digit m
164             6'd38: seg = 7'b1101110; // butt end of half of two digit m
165
166             default: seg = 7'b0000000; // default to blank for undefined codes
167         endcase
168     end
169 endmodule
170

```

Figure 14. Code snippet of Verilog file "AlphaTo7Segment" used to decode 6-bit numbers into 7-segment display code combinations.

```

245 // Binary to 7 segment conversion for the counts module calling
246 AlphaTo7Segment S1(.x(segIn1), .seg(seg1));
247 AlphaTo7Segment S2(.x(segIn2), .seg(seg2));
248 AlphaTo7Segment M1(.x(segIn3), .seg(seg3));
249 AlphaTo7Segment M2(.x(segIn4), .seg(seg4));
250 AlphaTo7Segment S11(.x(segIn5), .seg(seg5));
251 AlphaTo7Segment S22(.x(segIn6), .seg(seg6));
252 AlphaTo7Segment M11(.x(segIn7), .seg(seg7));
253 AlphaTo7Segment M22(.x(segIn8), .seg(seg8));

```

Figure 15. Code snippet of Verilog Module "AlphaTo7Segment" being initiated inside "THE_EGG_TIMER" eight times. Therefore, each digit has a decoder.

7-Segment Display Driver

The logic for the 7-segment display driving signal 'an' is shown below in Fig. 16. The 'an' signal is an 8-bit active low signal that directs which digit on the board should receive the 7-segment display code from the decoders used by the clock. This design refreshes the 'an' signal every to the 7-segment displays at a rate of 500Hz and ensures the 'seg' output is assigned to the correct decoder constantly. This design is built upon the innovative draft of our design.

```

23 module Four_seg_display_controller_500Hz(
24     input Clk_500Hz,
25     output reg [7:0] an
26 );
27     parameter S0 = 3'b000, S1 = 3'b001, S2 = 3'b010, S3 = 3'b011, S4 = 3'b100;
28     reg [2:0] count;
29
30     always @(posedge Clk_500Hz)
31     begin
32         count <= count + 1;
33         case (count)
34             S0: an <= 8'b11111110;
35             S1: an <= 8'b11111101;
36             S2: an <= 8'b11111011;
37             S3: an <= 8'b11110111;
38             S4: begin
39                 an <= 8'b11111111;
40                 count <= S0;
41             end
42         endcase
43     end
44 endmodule

```

Figure 16. Code snippet of Verilog Module "Four_seg_display_controller_500Hz". Code outputs 'an' signal which, is used to drive which segments receive the seven-segment display code

Master Controller

The master control circuit in this system is the module 'THE_EGG_TIMER'. This module is responsible for receiving and debouncing user input signals at 5MHz, driving the clock countdown and setup logic at 1Hz frequency, and connecting said elements to a display which it also drives at 500Hz frequency. The first portion of this module sets up all the clocks and modules used in these processes and can be seen in Fig. 17 below.

```

22 module THE_EGG_TIMER(
23     input clock100MHz, enable, reset, btnCookTime, btnMinutes, btnSeconds, btnStart,
24     output reg [6:0] seg,
25     output [7:0] an,
26     output TimerEnable, TimerOn
27 );
28
29     // Wire inputs
30     wire clock5MHz;
31     wire [6:0] seg1, seg2, seg3, seg4; // For each of the four 7 segment
32     wire minutesEN, secondsEN, cookTimeEN;
33     wire activeRST;
34
35     // Register outputs
36     wire clk1Hz;
37
38     wire clk_500Hz;
39     reg [22:0] clk_count;
40     reg [3:0] countm2, countm1, counts1, counts2;
41     reg [3:0] countdownm2, countdownm1, countdowns2, countdowns1;
42     reg start, pause;
43     reg countdownDone;
44     reg pauseTimer;
45
46     // Convert 100MHz to 5MHz
47     clk_wiz_0 fiveMHz(.clk_out1(clock5MHz), .reset(pauseTimer), .clk_in1(clock100MHz));
48
49     // Convert 5MHz to 1Hz
50     clk_div_1Hz oneHz(.clk_5MHz(clock5MHz), .reset(pauseTimer), .clk_out_1Hz(clk1Hz));
51
52     // 500Hz clock refresh rate to have a constant output on the 7 segments
53     clk_div_500Hz Clk500Hz(.clk_5MHz(clock5MHz), .clk_out_500Hz(clk_500Hz));

```

Figure 17. Code snippet of Verilog file Egg_timer.v initializing the "THE_EGG_TIMER" module and its inputs, outputs, connecting wires, and registers.

The second portion of this module is a query made using behavioural code at 5MHz frequency that checks the current state of the count-down clock based on the inputs and the 'countdownDone' variable. This section then drives the rest of the circuit components using the start and pause variables. The reset mechanism of the circuit is also controlled by lines 57 through 62. The button signals are debounced on the lines shortly below as explained in the debouncing subsection in this report section. After the signals are debounced, the logic shown in Fig. 8 through Fig. 11 controls the clock increments when set, and the clock de-increments when running. Logic controlling the count executes in the master controller, the clock digits are mapped to their respective decoders as shown in Fig. 13 of this report

```

55 : //Check if count down enabled, and running or done running
56 : always @(posedge clock5MHz) begin
57 :     if(reset) begin
58 :         // Reset states
59 :         start <= 0;
60 :         pause <= 0;
61 :         countdownDone <= 0;
62 :     end else if (enable) begin
63 :         if(countdowns1 == 0 && countdowns2 == 0 && countdownm1 == 0 && countdownm2 == 0 && start == 1)
64 :             countdownDone <= 1;
65 :     end
66 :     if(btnStart) begin
67 :         start <= 1;
68 :         pause <= 0;
69 :     end
70 :     if(cookTimeEN && start) begin
71 :         start <= 0;
72 :         pause <= 1;
73 :     end
74 : end
75 :
76 : //Debounce the minutes, seconds, and cookTime buttons
77 : Debounce mins(.clock5MHz(clock5MHz), .btn_pressed(btnMinutes), .btn_debounce(minutesEN));
78 : Debounce secs(.clock5MHz(clock5MHz), .btn_pressed(btnSeconds), .btn_debounce(secondsEN));
79 : Debounce cookTime(.clock5MHz(clock5MHz), .btn_pressed(btnCookTime), .btn_debounce(cookTimeEN));
80 :

```

Figure 18. Code snippet of Verilog file *Egg_timer.v* querying the current state of the clock and debouncing the input button signals.

The final task of the master controller is to initialize the 7-segment display driver which outputs the ‘an’ signal and then drive the correct segment values to the 7-segments based on the current value of ‘an’. This is shown below in Fig. 19 and the code controlling which digits decoded value is driven to the board is highlighted for clarity.

```

204 : Four_seg_display_controller_500Hz DC1(.Clk_500Hz(clk_500Hz), .an(an));
205 :
206 : // Updates the 7 segments everytime the counts are changed
207 : always @(an or seg1 or seg2 or seg3 or seg4) begin
208 :     case(an)
209 :         8'b11111110 : seg = seg1;
210 :         8'b11111101 : seg = seg2;
211 :         8'b11111011 : seg = seg3;
212 :         8'b11110111 : seg = seg4;
213 :         default : seg = 8'b11111111;
214 :     endcase
215 : end
216 :
217 : endmodule

```

Figure 19. Code snippet of Verilog file *Egg_timer* initializing the 7-segment display controller and defining the logic for the ‘seg’ output variable based on the current value of ‘an’.

Innovative and Alternative Solutions

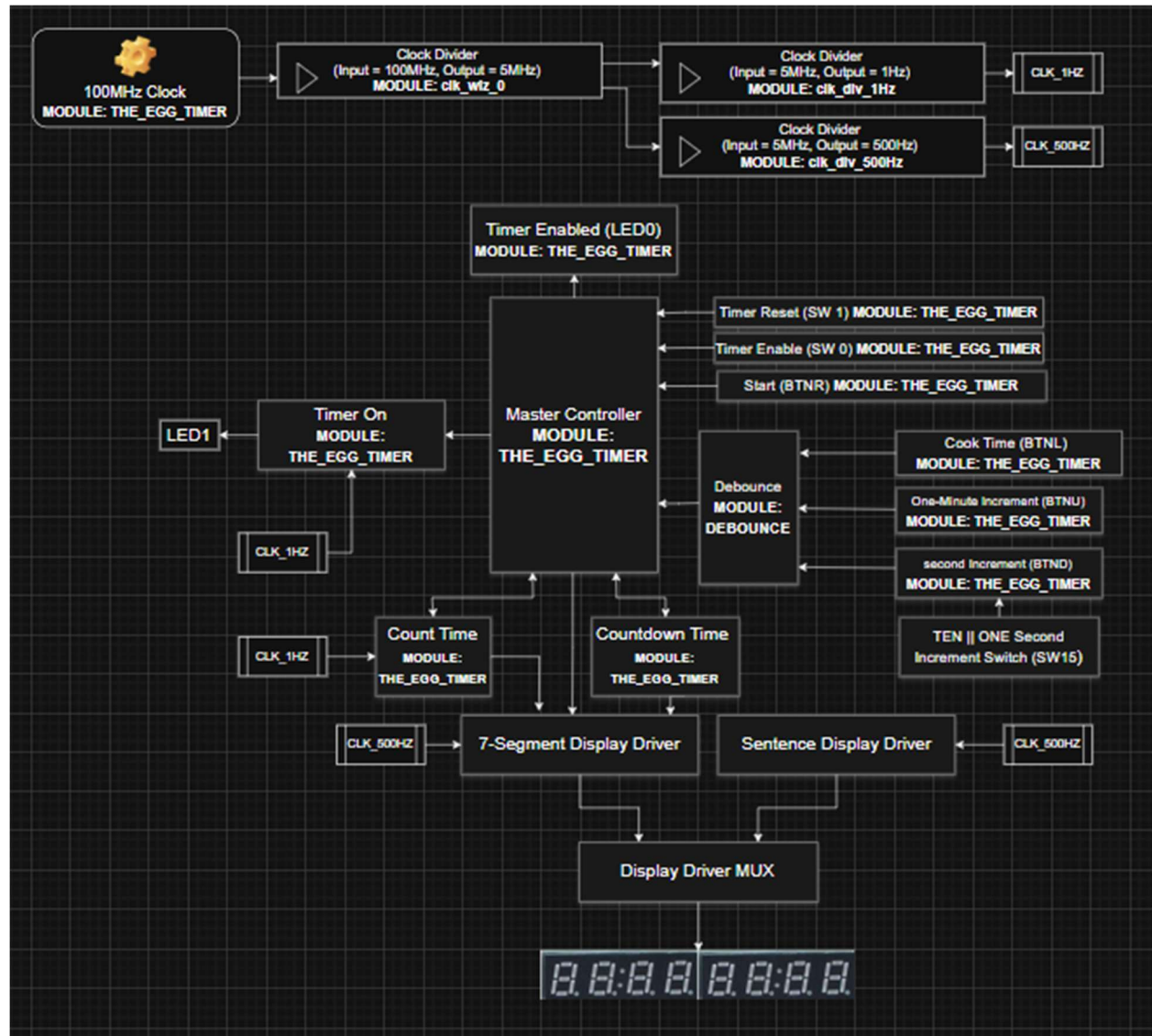


Figure 20. Egg timer system diagram with innovations made

Additional features were added to egg-timer to increase the functionality and UX experience of the user, however, not all these features were successfully integrated due to time constraints. The systems diagram in Fig. 20 illustrates the additional systems added. These additions include a toggle switch for the seconds count to increment one second or ten seconds as seen in Fig. 9 and discussed in the Count Time subsection of this report. This innovative feature should have been implemented on the minutes count but was not due to time constraints. The other innovative feature added to this design was the addition of a mechanism to alert the user when the clock has expired. This new feature required a 6-bit alpha-numerical decoder (Fig. 13), a message controller to encode and switch between sentences upon clock expiration, and a new display driver to drive eight 7-segment displays instead of four. This additional circuitry also required a redesign of the master controller which can be seen below in Fig. 22


```

86 module MessageController (
87     input wire clk_500Hz,
88     input wire countDone,
89     output reg messageSelect // 0 = first message ("ALL DONE"), 1 = second message ("GOODBYE")
90 );
91     localparam integer INTERVAL_TICKS = 5000; // 10 seconds worth of 500Hz ticks
92     reg [12:0] tickCount = 13'd0; // 13-bit counter (needs to count up to 5000)
93
94     always @(posedge clk_500Hz) begin
95         if (!countDone) begin
96             // If countDone is low, reset to initial state
97             tickCount <= 13'd0;
98             messageSelect <= 1'b0; // default to first message "ALL DONE"
99         end
100        else begin
101            // countDone is high: count ticks and toggle message every 10 seconds
102            if (tickCount >= INTERVAL_TICKS-1) begin
103                // Reached 5000 ticks (approximately 10 sec)
104                tickCount <= 13'd0;
105                messageSelect <= ~messageSelect; // toggle between 0 and 1
106            end
107            else begin
108                tickCount <= tickCount + 1'b1;
109            end
110        end
111    end
112 endmodule

```

Figure 21. Code snippet of Verilog file *sente3nceToSevenSegDecoder.v* defining the 'MessageController' module and defining the logic to alternate between completion messages.

The MessageController module (Fig. works by dividing the 500Hz clock signal down to 1/10Hz upon the 'countDone' signal being HI. When the 1/10Hz signal is LO the first message is displayed, and when it is HI the second message is displayed. The first output will always be LO so the sentences are displayed in order upon expiry.

The Display controller below named 'DisplayController8Digit' (Fig. 22) is responsible for generating the 'an' signal to control the eight 7-segment displays upon clock expiry. It also contains the logic to generate the correct 7-segment display code depending on the value of 'an'. This is done by pre-encoding the output 'letter_code' characters that make up a sentence in two set of 8 different states. Using the 'messageSelect' signal made by the message controller as the select input to a 2-1 MUX, the output sentence oscillates upon clock expiration. The current state is determined by the current value of 'an' and is output at the same time.

The master controller has had major additions from lines 220 to 273 (Fig. 23) to accomplish the output feedback. Lines 220 to 243 now change the value of the value passed to the alpha-numerical decoders based on the value of 'countdownDone'. If countdownDone is LO, the clock countdown digit signals are passed to the decoders, otherwise the sentence signals are passed to the decoders. On line 256 (Fig.23), a MUX between the clock display driver and sentence display driver signals using the countdownDone signal as the select signal. Finally, lines 258 to 270 have been altered to now always update all eight 7-segment displays every 500Hz. The innovative Verilog project files including the XDC files can be found in the second appendix of this report.


```

20 //////////////////////////////////////////////////
21 module DisplayController8Digit (
22     input wire clk_500Hz,
23     input wire countDone,
24     input wire messageSelect,
25     output reg [4:0] letter_code, // 0 or 1, from MessageController
26     output reg [7:0] an // Anode enables for 8 digits (active-low) // Segment outputs for LED segments a-g (active-low)
27 );
28 parameter S0 = 4'b0000, S1 = 4'b0001, S2 = 4'b0010, S3 = 4'b0011, S4 = 4'b0100, S5 = 4'b0101, S6 = 4'b0110, S7 = 4'b0111, S8 = 4'b1000;
29 reg [3:0] count; // scan index for current active digit (0-7)
30
31 always @(posedge clk_500Hz)
32 begin
33     count <= count + 1;
34     case (count)
35         S0: an <= 8'b11111110;
36         S1: an <= 8'b11111101;
37         S2: an <= 8'b11111011;
38         S3: an <= 8'b11110111;
39         S4: an <= 8'b11011111;
40         S5: an <= 8'b11011111;
41         S6: an <= 8'b10111111;
42         S7: an <= 8'b01111111;
43         S8: begin
44             an <= 8'b11111111;
45             count <= S0;
46         end
47     endcase
48 end
49
50 // 8-character message definitions (letter codes for each position)
51 // Using combinational logic to select the appropriate character for the active digit
52 always @(*) begin
53     if (messageSelect == 1'b0) begin
54         // Message 0: "ALL DONE" (8 characters including space)
55         case (count)
56             S0: letter_code = 6'd10; // A
57             S1: letter_code = 6'd11; // L
58             S2: letter_code = 6'd11; // L
59             S3: letter_code = 6'd36; // (space)
60             S4: letter_code = 6'd13; // D
61             S5: letter_code = 6'd24; // O
62             S6: letter_code = 6'd23; // N
63             S7: letter_code = 6'd14; // E
64         endcase
65     end
66     else begin
67         // Message 1: "GOODBYE " (7 letters + trailing space)
68         case (count)
69             S0: letter_code = 6'd16; // G
70             S1: letter_code = 6'd24; // O
71             S2: letter_code = 6'd24; // O
72             S3: letter_code = 6'd13; // D
73             S4: letter_code = 6'd11; // B
74             S5: letter_code = 6'd34; // Y
75             S6: letter_code = 6'd14; // E
76             S7: letter_code = 6'd36; // (space)
77         endcase
78     end
79 end
80 end
81 end
82
83 endmodule

```

Figure 22. Code snippet of Verilog file *sente3nceToSevenSegDecoder.v* defining the 'DisplayControllerDigit' module and encoding characters for the messages oscillating on the display upon clock expiry.

```

220     always @(*) begin
221         if (!countdownDone) begin
222             segin1 = countdowns1;
223             segin2 = countdowns2;
224             segin3 = countdownm1;
225             segin4 = countdownm2;
226             segin5 = 7'b0000000;
227             segin6 = 7'b0000000;
228             segin7 = 7'b0000000;
229             segin8 = 7'b0000000;
230         end
231         if (countdownDone) begin
232             case(an_sentence)
233                 8'b11111110 : segin1 = letter_code;
234                 8'b11111101 : segin2 = letter_code;
235                 8'b11111011 : segin3 = letter_code;
236                 8'b11110111 : segin4 = letter_code;
237                 8'b11101111 : segin5 = letter_code;
238                 8'b11011111 : segin6 = letter_code;
239                 8'b10111111 : segin7 = letter_code;
240                 8'b01111111 : segin8 = letter_code;
241             endcase
242         end
243     end
244
245     // Binary to 7 segment conversion for the counts module calling
246     AlphaTo7Segment S1(.x(segin1), .seg(seg1));
247     AlphaTo7Segment S2(.x(segin2), .seg(seg2));
248     AlphaTo7Segment M1(.x(segin3), .seg(seg3));
249     AlphaTo7Segment M2(.x(segin4), .seg(seg4));
250     AlphaTo7Segment S11(.x(segin5), .seg(seg5));
251     AlphaTo7Segment S22(.x(segin6), .seg(seg6));
252     AlphaTo7Segment M11(.x(segin7), .seg(seg7));
253     AlphaTo7Segment M22(.x(segin8), .seg(seg8));
254
255
256     assign an = (countdownDone) ? an_sentence : an_timer;
257
258     always @(an or seg1 or seg2 or seg3 or seg4 or seg5 or seg6 or seg7 or seg8) begin
259         case(an)
260             8'b11111110 : seg = seg1;
261             8'b11111101 : seg = seg2;
262             8'b11111011 : seg = seg3;
263             8'b11110111 : seg = seg4;
264             8'b11101111 : seg = seg5;
265             8'b11011111 : seg = seg6;
266             8'b10111111 : seg = seg7;
267             8'b01111111 : seg = seg8;
268             default      : seg = 7'b0000000;
269         endcase
270     end
271
272
273 endmodule

```

Figure 23. Code snippet of Verilog file *Egg_timer.v* showing the innovative changes made to the master controller module 'THE_EGG_TIMER'.

Observations

After the Egg Timer without innovations had been tested and verified, the innovative circuit could not pass the same tests regarding the function of the sentences displaying correctly upon clock expiration. Due to a lack of time, further tests could not be performed to solve the issue, but it was isolated. The eight 7-segment displays oscillated between two different sentences upon the clock expiry, but they were not made up of the correct characters. The 7-segment display codes for the character were tested by hand and some of the characters would work inconsistently. This led my team to determine a register of incorrect size was being used, or that there was a timing issue. Further testing is recommended to solve the issue. It is also recommended that my team implement the logic used in the second increment ten-second toggle switch to the minute increment portion of code for more ease of use. Preset times could also be implemented very easily for different stages of egg cooking (i.e. soft, medium, or hard-boiled) and mapped to the remaining switches on the board. The time constraints associated with this lab did not allow for this, however.

Conclusion

In this lab, the different concepts taught in the ELEC3500 were used extensively. These concepts included both up and down counters, synchronous and asynchronous circuit design, behavioral and structural, clock dividers, and more. The egg timer was designed to improve ease of use and user experience but could have timed and set eggs to cook more effectively by implementing preset times.

Although some of the egg-timer's additional features could not be implemented fully, the majority of the features did work and with access to more testing, the problems with the sentence display could be fixed promptly.

Acronyms and Terms

HI – Binary state of 1

LO – Binary state of 0

Button Spamming – Repetitive or random pressing of buttons on a controller or board

MUX – Short for Multiplexer

Signal Bounce – Switch behavior upon closure, resulting in output oscillating or rapidly changing before settling into a stable state

Appendix 1: No Innovation Egg Timer Full Verilog Code Files

Figure A.1 – Verilog file Egg_Timer.v. Part of the egg timer project without innovations.

```

timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 03/29/2025 12:46:22 PM
// Design Name:
// Module Name: Debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
module THE_EGG_TIMER(
    input clock100MHz, enable, reset, btnCookTime, btnMinutes, btnSeconds, btnStart,
    output reg [6:0] seg,
    output [7:0] an,
    output TimerEnable, TimerOn
);
// Wire inputs
wire clock5MHz;
wire clk_500Hz;
wire clk1Hz;
wire [6:0] seg1, seg2, seg3, seg4; // For each of the four 7 segment
wire minutesEN, secondsEN, cookTimeEN;
wire activeRST;

// Register outputs
reg [22:0] clk_count;
reg [3:0] countm2, countm1, counts1, counts2;
reg [3:0] countdownm2, countdownm1, countdowns2, countdowns1;
reg start, pause;
reg countdownDone;
reg pauseTimer;

// Convert 100MHz to 5MHz
clk_wiz_0_fiveMHz(.clk_out1(clock5MHz),.reset(pauseTimer),
.clk_in1(clock100MHz));

// Convert 5MHz to 1Hz
clk_div_1Hz oneHz(.clk_5MHz(clock5MHz),.reset(pauseTimer),
.clk_out_1Hz(clk1Hz));

//Convert 5MHz to 500Hz
clk_div_500Hz Clk500Hz(.clk_5MHz(clock5MHz), .clk_out_500Hz(clk_500Hz));

//Check if count down enabled, and running or done running
always @(posedge clock5MHz) begin
    if(reset) begin
        // Reset states
        clk_count <= 0;
        start <= 0;
        pause <= 0;
        countdownDone <= 0;
    end else if (enable) begin
        if(countdowns1 == 0 && countdowns2 == 0 && countdownm1 == 0 &&
countdownm2 == 0 && start == 1)
            countdownDone <= 1;
        end
        //else if (countdownDone) begin
        //countdowns1 <= 0; countdowns2 <= 0; countdownm1 <= 0; countdownm2 <=
0; start <= 1;
        //countdownDone <= 0;
        //end
        if(btnStart) begin
            start <= 1;
            pause <= 0;
        end
        if(cookTimeEN && start) begin
            start <= 0;
            pause <= 1;
        end
    end

    //Debounce the minutes, seconds, and cookTime buttons
    Debounce mins(.clock5MHz(clock5MHz), .btn_pressed(btnMinutes),
.btn_debounce(minutesEN));
    Debounce secs(.clock5MHz(clock5MHz), .btn_pressed(btnSeconds),
.btn_debounce(secondsEN));
    Debounce cookTime(.clock5MHz(clock5MHz), .btn_pressed(btnCookTime),
.btn_debounce(cookTimeEN));

    //Increment digits when setting the time. Display Format = M2:M1:S2:S1
    //Incrementing the minutes count
always@(posedge minutesEN or posedge reset) begin
    if(reset)begin
        countm2 <= 0;
        countm1 <= 0;
    end else if(btnCookTime && enable) begin
        // Minute 2 increment
        if(minutesEN && countm2 != 5 && countm1 == 9)
            countm2 <= countm2 + 1;
        else if(minutesEN && countm2 == 5 && countm1 == 9)
            countm2 <= 0;
        //Minute 1 increment
        if(minutesEN && countm1 != 9)
            countm1 <= countm1 + 1;
        else if(minutesEN && countm1 == 9)
            countm1 <= 0;
    end
end

// Incrementing the seconds count
// Timer format is M2:M1:S2:S1
always@(posedge secondsEN or posedge reset) begin
    if(reset)begin
        counts2 <= 0;
        counts1 <= 0;
    end else if(btnCookTime && enable) begin
        //Second 2 increment
        if(secondsEN && counts2 != 5 && counts1 == 9)
            counts2 <= counts2 + 1;
        else if(secondsEN && counts2 == 5 && counts1 == 9)
            counts2 <= 0;
        // Second 1 increment
        if(secondsEN && counts1 != 9)
            counts1 <= counts1 + 1;
        else if(secondsEN && counts1 == 9)
            counts1 <= 0;
    end
end

//De-increment when timer started
//Seconds 1 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdowns1 <= 4'b0000;
    else begin
        if(!start && !pause)
            countdowns1 <= counts1;
        else if(pause)
            countdowns1 <= countdowns1;
        if(start && !countdownDone) begin
            if(countdowns1 == 4'b0000)
                countdowns1 <= 4'b1001;
            else
                countdowns1 <= countdowns1 - 1;
        end
    end
end

//Seconds 2 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdowns2 <= 4'b0000;
    else begin
        if(!start && !pause)
            countdowns2 <= counts2;
        else if(pause)
            countdowns2 <= countdowns2;
        if(start && !countdownDone) begin
            if(countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
                countdowns2 <= 4'b0101;
            else if(countdowns1 == 4'b0000)
                countdowns2 <= countdowns2 - 1;
        end
    end
end

//Minutes 1 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdownm1 <= 4'b0000;
    else begin
        if(!start && !pause)
            countdownm1 <= countm1;
        else if(pause)
            countdownm1 <= countdownm1;
        if(start && !countdownDone) begin
            if(countdownm1 == 4'b0000 && countdowns2 == 4'b0000 && countdowns1
== 4'b0000)
                countdownm1 <= 4'b1001;
            else if(countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
                countdownm1 <= countdownm1 - 1;
        end
    end
end
end
end

```

```

//Minutes 2 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdown2 <= 4'b0000;
    else begin
        if(!start && !pause)
            countdown2 = countm2;
        else if(pause)
            countdown2 <= countdown2;
        if(start && !countdownDone) begin
            if(countdownm1 == 4'b0000 && countdowns2 == 4'b0000 && countdowns1
== 4'b0000)
                countdown2 <= countdown2 - 1;
            end
        end
    end
end

// Output wires to display on LEDS
assign TimerEnable = enable; // LED0
assign TimerOn = (start && enable) ? 1'b1: 1'b0; // LED1

// Active register for reset
assign activeRST = (reset == 1) ? 1'b1: 1'b0;

// Binary to 7 segment conversion for the counts module calling
bcdto7segment_dataflow S1(.x(countdowns1), .seg(seg1));
bcdto7segment_dataflow S2(.x(countdowns2), .seg(seg2));
bcdto7segment_dataflow M1(.x(countdownm1), .seg(seg3));
bcdto7segment_dataflow M2(.x(countdownm2), .seg(seg4));

Four_seg_display_controller_500Hz DC1(.Clk_500Hz(clk_500Hz), .an(an));

// Updates the 7 segments everytime the counts are changed
always @(an or seg1 or seg2 or seg3 or seg4) begin
    case(an)
        8'b11111110 : seg = seg1;
        8'b11111101 : seg = seg2;
        8'b11111011 : seg = seg3;
        8'b11110111 : seg = seg4;
        default : seg = 8'b11111111;
    endcase
end
endmodule

```

Figure A.2 – Verilog file Clock_Divider.v. Part of the egg timer project without innovations

```

module clk_div_500Hz (
    input wire clk_5MHz,
    input wire reset,
    output reg clk_out_500Hz
);

    reg [20:0] count; // 13-bit counter for counting 5000 ticks

    always @(posedge clk_5MHz) begin
        if (reset) begin
            count <= 0;
            clk_out_500Hz <= ~clk_out_500Hz;
        end else begin
            if (count == 4999) begin
                clk_out_500Hz <= ~clk_out_500Hz;
                count <= 0;
            end else begin
                count <= count + 1;
            end
        end
    end
endmodule

module clk_div_1Hz (
    input wire clk_5MHz,
    input wire reset,
    output reg clk_out_1Hz
);

    reg [22:0] count; // 13-bit counter for counting 5000 ticks

    always @(posedge clk_5MHz or posedge reset) begin
        if (reset) begin
            count <= 0;
            clk_out_1Hz <= ~clk_out_1Hz;
        end else begin
            if (count == 4999999/2) begin
                clk_out_1Hz <= ~clk_out_1Hz;
                count <= 0;
            end else begin
                count <= count + 1;
            end
        end
    end
endmodule

```

Figure A.3 – Verilog file sevenSegBCDDisplay.v. Part of the egg timer project without innovations

```

module bcdto7segment_dataflow(
    input wire [3:0] x,
    output reg [6:0] seg
);

always @ (*) begin
    case (x)
        4'b0000: seg <= 7'b1000000;
        4'b0001: seg <= 7'b1111001;
        4'b0010: seg <= 7'b0100100;
        4'b0011: seg <= 7'b0110000;
        4'b0100: seg <= 7'b0011001;
        4'b0101: seg <= 7'b0010010;
        4'b0110: seg <= 7'b0000010;
        4'b0111: seg <= 7'b1111000;
        4'b1000: seg <= 7'b0000000;
        4'b1001: seg <= 7'b0010000;
        default: seg <= 7'b1111111;
    endcase
end
endmodule

```

Figure A.4 – Verilog file DisplayController.v. Part of the egg timer project without innovations

```

module Four_seg_display_controller_500Hz(
    input Clk_500Hz,
    output reg [7:0] an
);
    parameter S0 = 3'b000, S1 = 3'b001, S2 = 3'b010, S3 = 3'b011, S4 = 3'b100;
    reg [2:0] count;

    always @(posedge Clk_500Hz)
        begin
            count <= count + 1;
            case (count)
                S0: an <= 8'b11111110;
                S1: an <= 8'b11111101;
                S2: an <= 8'b11111011;
                S3: an <= 8'b11110111;
                S4: begin
                    an <= 8'b11111111;
                    count <= S0;
                end
            endcase
        end
end
endmodule

```

Figure A.5 – Verilog file Debounce.v. Part of the egg timer project without innovations

```

module Debounce(
    input clock5MHz,
    input btn_pressed,
    output reg btn_debounce
);
    reg store_value;
    reg [22:0] high_count, low_count;

    always@(posedge clock5MHz) begin
        store_value <= btn_pressed;
        if(high_count == 50000 || low_count == 50000) begin
            btn_debounce <= store_value;
            high_count <= 0;
            low_count <= 0;
        end else if(btn_pressed)
            high_count <= high_count + 1;
        else
            low_count <= low_count + 1;
    end
end
endmodule

```


Appendix 2: Innovation Egg Timer Full Verilog Code Files

Figure A.6 – Verilog file Egg_Timer.v. Part of the egg timer project with innovations.

```

`timescale 1ns / 1ps
// Company:
// Engineer:
//
// Create Date: 03/29/2025 12:46:22 PM
// Design Name:
// Module Name: Debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//
module THE_EGG_TIMER(
    input clock100MHz, enable, reset, btnCookTime, btnMinutes, btnSeconds, btnStart,
    output reg [6:0] seg,
    output [7:0] an,
    output TimerEnable, TimerOn
);

// Wire inputs
wire clock5MHz;
wire [6:0] seg1, seg2, seg3, seg4, seg5, seg6, seg7, seg8; // For each of the
// 8 segment
reg [5:0] segIn1, segIn2, segIn3, segIn4, segIn5, segIn6, segIn7, segIn8;
wire [5:0] letter_code;
wire minutesEN, secondsEN, cookTimeEN;
wire activeRST;

// Register outputs
wire clk1Hz;
reg [22:0] clk_count;
reg [3:0] counts2, countml, counts1, counts2;
reg [3:0] countdown2, countdownml, countdowns2, countdowns1;
reg start, pause;
reg countdownDone;
reg pauseTimer;

// Convert 100MHz to 5MHz
clk_wiz_0_fiveMHz(.clk_out1(clock5MHz), .reset(reset), .clk_in1(clock100MHz));

// Convert 5MHz to 1Hz
clk_div_1Hz_oneHz(.clk_5MHz(clock5MHz), .reset(reset), .clk_out_1Hz(clk1Hz));

// Check if count down enabled, and running or done running
always @(posedge clock5MHz) begin
    if(reset) begin
        // Reset states
        clk_count <= 0;
        start <= 0;
        pause <= 0;
        countdownDone <= 0;
    end else if (enable) begin
        if(countdowns1 == 0 && countdowns2 == 0 && countdownml == 0 &&
        countdown2 == 0 && start == 1)
            countdownDone <= 1;
    end

    if(btnStart) begin
        start <= 1;
        pause <= 0;
    end

    if(cookTimeEN && start) begin
        start <= 0;
        pause <= 1;
    end
end

//Debounce the minutes, seconds, and cookTime buttons
Debounce mins(.clock5MHz(clock5MHz), .btn_pressed(btnMinutes),
.btn_debounce(minutesEN));
Debounce secs(.clock5MHz(clock5MHz), .btn_pressed(btnSeconds),
.btn_debounce(secondsEN));
Debounce cookTime(.clock5MHz(clock5MHz), .btn_pressed(btnCookTime),
.btn_debounce(cookTimeEN));

//Increment digits when setting the time. Format = M2:M1:S2:S1
//Incrementing the minutes count
always @(posedge minutesEN or posedge reset) begin
    if(reset)begin
        counts2 <= 0;
        countml <= 0;
    end else if(btnCookTime && enable) begin
        // Minute 2 increment
        if(minutesEN && counts2 != 5 && countml == 9)
            counts2 <= counts2 + 1;
        else if(minutesEN && counts2 == 5 && countml == 9)
            counts2 <= 0;
        //Minute 1 increment
        if(minutesEN && countml != 9)
            countml <= countml + 1;
        else if(minutesEN && countml == 9)
            countml <= 0;
    end
end

// Incrementing the seconds count
// Timer format is M2:M1:S2:S1
always @(posedge secondsEN or posedge reset) begin
    if(reset)begin
        counts2 <= 0;
        counts1 <= 0;
    end else if(btnCookTime && enable) begin
        //Second 2 increment
        if(secondsEN && counts2 != 5 && counts1 == 9)
            counts2 <= counts2 + 1;
        else if (secondsEN && counts2 != 5 && !countTens)
            counts2 <= counts2 + 1;
        else if (secondsEN && counts2 == 5 && !countTens)
            counts2 <= 0;
        else if(secondsEN && counts2 == 5 && counts1 == 9)
            counts2 <= 0;
        // Second 1 increment
        if(secondsEN && counts1 != 9 && !countTens)
            counts1 <= counts1 + 1;
        else if(secondsEN && counts1 == 9 && !countTens)
            counts1 <= 0;
    end
end

//De-increment when timer started
//Seconds 1 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdowns1 <= 4'b0000;
    else begin
        if(!start && !pause)
            countdowns1 <= counts1;
        else if(pause)
            countdowns1 <= countdowns1;
        if(start && !countdownDone) begin
            if(countdowns1 == 4'b0000)
                countdowns1 <= 4'b1001;
            else
                countdowns1 <= countdowns1 - 1;
        end
    end
end

//Seconds 2 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdowns2 <= 4'b0000;
    else begin
        if (!start && !pause)
            countdowns2 <= counts2;
        else if(pause)
            countdowns2 <= countdowns2;
        if(start && !countdownDone) begin
            if(countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
                countdowns2 <= 4'b0101;
            else if(countdowns1 == 4'b0000)
                countdowns2 <= countdowns2 - 1;
        end
    end
end

//Minutes 1 De-increment
always @(posedge clk1Hz) begin
    if(activeRST)
        countdownml <= 4'b0000;
    else begin
        if(!start && !pause)
            countdownml = countml;
        else if(pause)
            countdownml <= countdownml;
        if(start && !countdownDone) begin
            if(countdownml == 4'b0000 && countdowns2 == 4'b0000 && countdowns1
            == 4'b0000)
                countdownml <= 4'b1001;
            else if(countdowns2 == 4'b0000 && countdowns1 == 4'b0000)
                countdownml <= countdownml - 1;
        end
    end
end
end

```

```

//Minutes 2 De-increment
always @(posedge clkHz) begin
    if(activeRST)
        countdownm2 <= 4'b0000;
    else begin
        if(!start && !pause)
            countdownm2 = countm2;
        else if(pause)
            countdownm2 <= countdownm2;
        if(start && !countdownDone) begin
            if(countdownm1 == 4'b0000 && countdowns2 == 4'b0000 && countdowns1
== 4'b0000)
                countdownm2 <= countdownm2 - 1;
            end
        end
    end
end

// Output wires to display on LEDS
assign TimerEnable = enable; // LED0
assign TimerOn = (start && enable) ? 1'b1: 1'b0; // LED1

// Active register for reset
assign activeRST = (reset == 1) ? 1'b1: 1'b0;

// Wires for 8-digit sentence display controller
wire [7:0] an_sentence;
wire [6:0] seg_sentence;
wire [7:0] an_timer;
wire messageSelect;

// 500Hz clock refresh rate to have a constant output on the 7 segments
wire clk_500Hz;
clk_div_500Hz Clk500Hz(.clk_5MHz(clock5MHz), .reset(reset),
.clk_out_500Hz(clk_500Hz));

//Initiate Counter controller
Four_seg_display_controller_500Hz DC1(.Clk_500Hz(clk_500Hz), .an(an_timer));

//Initiate Sentence Controllerz
MessageController msgCtrl (.clk_500Hz(clk_500Hz), .countDone(countdownDone),
.messageSelect(messageSelect));
DisplayController8Digit sentenceDisp (.clk_500Hz(clock_500Hz),
.countDone(countdownDone), .messageSelect(messageSelect), .letter_code(letter_code),
.an(an_sentence));

always @(*) begin
    if (!countdownDone) begin
        segin1 = countdowns1;
        segin2 = countdowns2;
        segin3 = countdownm1;
        segin4 = countdownm2;
        segin5 = 7'b00000000;
        segin6 = 7'b00000000;
        segin7 = 7'b00000000;
        segin8 = 7'b00000000;
    end
    if (countdownDone) begin
        case(an_sentence)
            8'b11111110 : segin1 = letter_code;
            8'b11111101 : segin2 = letter_code;
            8'b11111011 : segin3= letter_code;
            8'b11110111 : segin4 = letter_code;
            8'b11011111 : segin5 = letter_code;
            8'b10111111 : segin6 = letter_code;
            8'b01111111 : segin7 = letter_code;
            8'b01111111 : segin8 = letter_code;
        endcase
    end
end

// Binary to 7 segment conversion for the counts module calling
AlphaTo7Segment S1(.x(segin1), .seg(seg1));
AlphaTo7Segment S2(.x(segin2), .seg(seg2));
AlphaTo7Segment M1(.x(segin3), .seg(seg3));
AlphaTo7Segment M2(.x(segin4), .seg(seg4));
AlphaTo7Segment S11(.x(segin5), .seg(seg5));
AlphaTo7Segment S22(.x(segin6), .seg(seg6));
AlphaTo7Segment M11(.x(segin7), .seg(seg7));
AlphaTo7Segment M22(.x(segin8), .seg(seg8));

assign an = (countdownDone) ? an_sentence : an_timer;

always @(an or seg1 or seg2 or seg3 or seg4 or seg5 or seg6 or seg7 or seg8)
begin
    case(an)
        8'b11111110 : seg = seg1;
        8'b11111101 : seg = seg2;
        8'b11111011 : seg = seg3;
        8'b11110111 : seg = seg4;
        8'b11011111 : seg = seg5;
        8'b10111111 : seg = seg6;
        8'b01111111 : seg = seg7;
        8'b01111111 : seg = seg8;
        default : seg = 7'b00000000;
    endcase
end
endmodule

```

Figure A.7 – Verilog file Clock_Divider.v. Part of the egg timer project with innovations

```

module clk_div_500Hz (
    input wire clk_5MHz,
    input wire reset,
    output reg clk_out_500Hz
);

    reg [20:0] count; // 13-bit counter for counting 5000 ticks

    always @(posedge clk_5MHz) begin
        if (reset) begin
            count <= 0;
            clk_out_500Hz <= ~clk_out_500Hz;
        end else begin
            if (count == 4999) begin
                clk_out_500Hz <= ~clk_out_500Hz;
                count <= 0;
            end else begin
                count <= count + 1;
            end
        end
    end
endmodule

module clk_div_1Hz (
    input wire clk_5MHz,
    input wire reset,
    output reg clk_out_1Hz
);

    reg [22:0] count; // 13-bit counter for counting 5000 ticks

    always @(posedge clk_5MHz or posedge reset) begin
        if (reset) begin
            count <= 0;
            clk_out_1Hz <= ~clk_out_1Hz;
        end else begin
            if (count == 4999999/2) begin
                clk_out_1Hz <= ~clk_out_1Hz;
                count <= 0;
            end else begin
                count <= count + 1;
            end
        end
    end
endmodule

```

Figure A.8 – Verilog file DisplayController.v. Part of the egg timer project with innovations

```

module Four_seg_display_controller_500Hz(
    input Clk_500Hz,
    output reg [7:0] an
);
    parameter S0 = 3'b000, S1 = 3'b001, S2 = 3'b010, S3 = 3'b011, S4 = 3'b100;
    reg [2:0] count;

    always @(posedge Clk_500Hz)
    begin
        count <= count + 1;
        case (count)
            S0: an <= 8'b11111110;
            S1: an <= 8'b11111101;
            S2: an <= 8'b11111011;
            S3: an <= 8'b11110111;
            S4: begin
                an <= 8'b11111111;
                count <= S0;
            end
        endcase
    end
endmodule

```

Figure A.9 – Verilog file Debounce.v. Part of the egg timer project without innovations

```

module Debounce(
    input clock5MHz,
    input btn_pressed,
    output reg btn_debounce
);
    reg store_value;
    reg [22:0] high_count, low_count;

    always@(posedge clock5MHz) begin
        store_value <= btn_pressed;
        if(high_count == 50000 || low_count == 50000) begin
            btn_debounce <= store_value;
            high_count <= 0;
            low_count <= 0;
        end else if(btn_pressed)
            high_count <= high_count + 1;
        else
            low_count <= low_count + 1;
        end
    end
endmodule

```

Figure A.10 – Verilog file sentenceToSevenDecoder.v. Part of the egg timer project with innovations.

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 03/29/2025 03:16:44 PM
// Design Name:
// Module Name: sentenceToSevenSegDecoder
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
module DisplayController8Digit (
    input wire clk_500Hz,
    input wire countDone,
    input wire messageSelect,
    output reg [4:0] letter_code, // 0 or 1, from MessageController
    output reg [7:0] an // Anode enables for 8 digits (active-low)
    // Segment outputs for LED segments a-g (active-low)
);
    parameter S0 = 4'b0000, S1 = 4'b0001, S2 = 4'b0010, S3 = 4'b0011, S4 = 4'b0100,
    S5 = 4'b0101, S6 = 4'b0110, S7 = 4'b0111, S8 = 4'b1000;
    reg [3:0] count; // scan index for current active digit (0-7)

    always @(posedge clk_500Hz)
    begin
        count <= count + 1;
        case (count)
            S0: an <= 8'b11111110;
            S1: an <= 8'b11111101;
            S2: an <= 8'b11111011;
            S3: an <= 8'b11110111;
            S4: an <= 8'b11101111;
            S5: an <= 8'b11011111;
            S6: an <= 8'b10111111;
            S7: an <= 8'b01111111;
            S8: begin
                an <= 8'b11111111;
                count <= S0;
            end
        endcase
    end

    // 8-character message definitions (letter codes for each position)
    // Using combinational logic to select the appropriate character for the active
    digit
    always @(*) begin
        if (messageSelect == 1'b0) begin
            // Message 0: "ALL DONE" (8 characters including space)
            case (count)
                S0: letter_code = 6'd10; // A
                S1: letter_code = 6'd11; // L
                S2: letter_code = 6'd11; // L
                S3: letter_code = 6'd36; // (space)
                S4: letter_code = 6'd13; // D
                S5: letter_code = 6'd24; // O
                S6: letter_code = 6'd23; // N
                S7: letter_code = 6'd14; // E
            endcase
        end
        else begin
            // Message 1: "GOODBYE " (7 letters + trailing space)
            case (count)
                S0: letter_code = 6'd16; // G
                S1: letter_code = 6'd24; // O
                S2: letter_code = 6'd24; // O
                S3: letter_code = 6'd13; // D
                S4: letter_code = 6'd11; // B
                S5: letter_code = 6'd34; // Y
                S6: letter_code = 6'd14; // E
                S7: letter_code = 6'd36; // (space)
            endcase
        end
    end
endmodule

module MessageController (
    input wire clk_500Hz,
    input wire countDone,
    output reg messageSelect // 0 = first message ("ALL DONE"), 1 = second message
);
    localparam integer INTERVAL_TICKS = 5000; // 10 seconds worth of 500Hz
    ticks
    reg [12:0] tickCount = 13'd0; // 13-bit counter (needs to
    count up to 5000)

    always @(posedge clk_500Hz) begin
        if (!countDone) begin
            // If countDone is low, reset to initial state
            tickCount <= 13'd0;
            messageSelect <= 1'b0; // default to first message "ALL DONE"
        end
        else begin
            // countDone is high: count ticks and toggle message every 10 seconds
            if (tickCount >= INTERVAL_TICKS-1) begin
                // Reached 5000 ticks (approximately 10 sec)
                tickCount <= 13'd0;
                messageSelect <= ~messageSelect; // toggle between 0 and 1
            end
            else begin
                tickCount <= tickCount + 1'b1;
            end
        end
    end
endmodule

// Decoder: 5-bit letter code (A-Z, space) to 7-segment pattern
module AlphaTo7Segment (
    input [6:0] x, // Letter code: 0=A,1=B,...25=Z, 26=blank
    output reg [6:0] seg // 7-segment output (active-low)
);
    always @(*) begin
        case (x)
            // A-Z letters:
            6'd0: seg <= 7'b1000000;
            6'd1: seg <= 7'b1111001;
            6'd2: seg <= 7'b1001001;
            6'd3: seg <= 7'b0110000;
            6'd4: seg <= 7'b0011001;
            6'd5: seg <= 7'b0010010;
            6'd6: seg <= 7'b0000010;
            6'd7: seg <= 7'b1111000;
            6'd8: seg <= 7'b0000000;
            6'd9: seg <= 7'b0010000;

            //now define teh charactors
            6'd10: seg <= 7'b0001000; // A -> segments a,b,c,e,f,g on (shape: A with
            middle, no bottom)
            6'd11: seg <= 7'b0011111; // B -> segments b,c,d,e,f,g on (approximate
            'B', looks like 8)
            6'd12: seg <= 7'b0110001; // C -> segments a,d,e,f on (looks like C)
            6'd13: seg <= 7'b0011011; // D -> segments b,c,d,e,g on (approximate
            'D', looks like 0)
            6'd14: seg <= 7'b0110011; // E -> segments a,d,e,f,g on (E)
            6'd15: seg <= 7'b0111000; // F -> segments a,e,f,g on (F)
            6'd16: seg <= 7'b0101111; // G -> segments a,c,d,e,f,g on (G)
            6'd17: seg <= 7'b0011100; // H -> segments b,c,e,f,g on (H)
            6'd18: seg <= 7'b0000110; // I -> segments b,c on (I, shown as l)
            6'd19: seg <= 7'b0011110; // J -> segments b,c,d,e on (J)
            6'd20: seg <= 7'b0010101; // K -> segments b,c,e,f,g on (approximate K as
            R, 7-seg can't display K distinctly)
            6'd21: seg <= 7'b0110000; // L -> segments d,e,f on (L)
            6'd22: seg <= 7'b1000111; // M -> segments b,c,e,f on (approximate M as
            two vertical columns)
            6'd23: seg <= 7'b0101011; // N -> segments c,e,g on lowercase (N)
            6'd24: seg <= 7'b0111111; // O -> segments a,b,c,d,e,f on (O, same as 0)
            6'd25: seg <= 7'b0011000; // P -> segments a,b,e,f,g on (P)
            6'd26: seg <= 7'b0011010; // Q -> segments a,b,c,d,f,g on (Q, looks like
            9)
            6'd27: seg <= 7'b0011001; // R -> segments a,b,c,e,f,g on (R, appears
            similar to A on 7-seg)
            6'd28: seg <= 7'b0100101; // S -> segments a,c,d,f,g on (S, like digit 5)
            6'd29: seg <= 7'b0001111; // T -> segments a,b,c on (T, shown as 7)
            6'd30: seg <= 7'b0011110; // U -> segments b,c,d,e,f on (U)
            6'd31: seg <= 7'b0011101; // V -> segments c,e on (V, appears as two
            bottom corners lit)
            6'd32: seg <= 7'b0011110; // W -> segments b,c,d,e,f on (W, shown as U)
            6'd33: seg <= 7'b0001001; // X -> segments b,c,e,f on (X, shown as two
            vertical columns like M)
            6'd34: seg <= 7'b0011010; // Y -> segments b,c,d,f,g on (Y)
            6'd35: seg <= 7'b0110010; // Z -> segments a,b,d,e,g on (Z, looks like
            digit 2)

            // Blank (space):
            6'd36: seg <= 7'b1111111; // all segments off (blank)
            6'd37: seg <= 7'b1101010; // front half of two digit m
            6'd38: seg <= 7'b1101110; // butt end of half of two digit m

            default: seg = 7'b0000000; // default to blank for undefined codes
        endcase
    end
endmodule

```